

Claude 아티팩트 분석

👤 작성자

박솔민(25반_4255)

| Claude Artifact Forensics — Measured Verification v2 | 2026-08-11

▼ 1. 검증 환경 및 방법

항목	내용
검증 대상 PC	Windows 11 Pro 26200 / x64
웹	Google Chrome 151.0.7922.108
앱	Claude Desktop MSIX Claude_1.24012.9.0_x64__pzs8srxrjxfjjc
사용 도구	ccl_chromium_reader 0.3.18 / ccl_simplesnappy 0.4 / zstandard / Python 3.11 sqlite3 / PowerShell
수집 방식	원본 경로 robocopy 사본 → 사본에 대해서만 파싱

▼ 2. 대화 데이터가 실제로 어디 남는지

데이터	실제 소재
대화 본문(사람·모델 발화 전문)	앱 HTTP Cache chat_conversations/<uuid>?tree=True (zstd)
대화 목록(UUID·제목·시각·모델)	웹·앱 IndexedDB keyval-store(react-query) + 양쪽 HTTP Cache chat_conversations_v2
미전송 입력(타이핑 과정)	앱 IndexedDB store:chat-draft:<uuid>
첨부 파일	IndexedDB Blob 저장소(OPFS 아님)
삭제 흔적	LevelDB tombstone — 앱 keyval 45건 · web 1건, chat-draft 43건

간단한 테스트 결과 IndexedDB에는 목록 메타데이터와 입력 초안만 있고, 본문은 HTTP 캐시에 zstd로 통째로 남는다. 조사 우선 순위: Cache → IndexedDB

▼ 3. 웹 브라우저 (Chrome) — claude.ai

분석 대상	저장 위치	도구	실측 결과
Web History	...\Default\History (urls · visits)	sqlite3 / DB Browser	claude.ai 접속 URL 8행 실측. /chat/<UUID> 형식은 0건 — 로우와 /new 만 기록. typed_ 부 0(주소창 직접입력 이력 없음) visit_duration 2,485초.
History WAL	...\Default\History-wal · -journal	DB Browser	본 프로파일은 journal 0바이트 (=DELETE 모드, WAL 미사용)는 것은 아니다.
Download	...\Default\History (downloads)	sqlite3	총 2건 모두 claude 무관. 구조: 나 본 사례 미검출.
Keyword Search	...\Default\History (keyword_search_terms)	sqlite3	term='클로드' @2026-07-31:26:09 — 검색 경유 접속을 2
Cookies	...\Default\Network\Cookies (DPAPI+AES-GCM)	sqlite3 + DPAPI 복호화	claude.ai 쿠키 16개 실측: sessionKey(httponly,194B) · sessionKeyLC · routingHint(cf_clearance · __cf_bm · lastActiveOrg · anthropic-de activitySessionId · ajs_anonymous_id · __dd_s · _gcl_au 등. 생성/만료/최종접근 두 보존.

분석 대상	저장 위치	도구	실측 결과
IndexedDB	... Default\IndexedDB\https_claude.ai_0.indexeddb.leveldb\	ccl_chromium_reader	DB 2개: keyval-store(store=173레코드 / live 172 · 삭제 1), device-binding(0건). keyval react-query 영속 스냅샷 170개 queryKey 7종 — chat_conversation_list / current_account / my_access_account_profile / subscription_details / sessions_api_list_sessions. 메타데이터(UUID·제목·생성/수정)는 있으나 메시지 본문은 없음
IndexedDB Blob	...\https_claude.ai_0.indexeddb.blob\1\00\	헥스 에디터	참조 레코드가 없는데도 blob 112,215,914 bytes 잔존. 복구?
Local Storage	...\Default\Local Storage\leveldb\	ccl_chromium_reader	claude.ai 91레코드. default_model=claude-opus-5, spa:locale=ko-KR, __qk_hint_account_uuid, cache-confirmed-account, sync-owner (계정 UUID), analytics_queue_v1 (행위 12,638B).
Session Storage	...\Default\Session Storage\	ccl_chromium_reader	host https://claude.ai/ 후 popup_google_auth_state, spa_nav_loop, claudeai.perf.interaction_slc SSS-* 배너 상태.
HTTP Cache	...\Default\Cache\Cache_Data\	ccl_chromium_reader	blockfile 백엔드(data_0~3 · 분리 814건 중 claude 336건. Af재: chat_conversations_v2(5account_profile, subscription_details, cowork_settings, mcp/remote_servers 등 137개 zstd 압축.
Service Worker	...\Default\Service Worker\ScriptCache\	ccl_chromium_reader	디렉터리·엔트리 존재하나 claude 속 여부 미확인.
Session / Tabs	...\Default\Sessions\Session_<n> · Tabs_<n>	SNSS 파서	Chrome 151에는 Current Session Current Tabs · Last Session Tabs 파일이 없다. Sessions\Session_ / Tabs_ 만 존재.
Bookmarks	...\Default\Bookmarks	텍스트 에디터	claude 항목 0건. 구조상 가능하리라 미검출.
Shortcuts / Top Sites	...\Default\Shortcuts · Top Sites	sqlite3	claude 항목 0건. typed_count(주소창 입력 이력 없음).
DIPS	...\Default\DIPS (bounces)	sqlite3	claude.ai / claude.com의 사용' 최초·최종 시각이 마이크로초로 기록. 단순 로드가 아닌 실제명.
Quota Buckets	...\Default\WebStorage\QuotaManager (buckets)	sqlite3	https://claude.ai/ bucket use_count=20, last_accessed 07-31 12:08:23 — History 마지막 (11:26)보다 42분 늦다. SPA 체 History 없이 증명. 파티션 키 nocookie.com/^0https://claude도 관측.

▼ 4. 애플리케이션 (Windows) — Claude Desktop

분석 대상	저장 위치	도구
1 설치 형태	Get-AppxPackage → <code>Claude_1.30096.1.0_x64__pzs8srxjxfjjc</code>	PowerShell
2 데이터 루트	<code>%LOCALAPPDATA%\Packages\Claude_pzs8srxjxfjjc\LocalCache\Roaming\Claude\</code>	탐색기
3 IndexedDB	<code>...\Claude\IndexedDB\https_claude.ai_0.indexeddb.leveldb\</code>	ccl_chromium_reader
4 입력 초안	<code>keyval-store</code> → <code>store:chat-draft:<대화식별자></code> 대화 생성 전 입력은 고정 키 <code>store:chat-draft:chorus-unified-composer</code> 값이 큰 초안은 <code>https_claude.ai_0.indexeddb.blob</code> 폴더에 외부 파일로 분리	<code>ccl_chromium_reader</code> (WrappedRawLevelDb)
5 대화 첨부 파일 (blob)	위치① <code>keyval-store</code> → <code>store:attachment-blob:<uuid></code> — 포인터② <code>...indexeddb.blob\{db}\{번호>>8:02x}\{번호:x}</code> — 실물③ <code>Roaming\Claude\pending-uploads\</code> — 업로드 스테이징 원본(평문 PNG)	<code>ccl_chromium_reader</code> (WrappedRawLevelDb) <code>resolve_blob_index</code> , <code>get_blob_s</code>
6 HTTP Cache	<code>...\Claude\Cache\Cache_Data\</code> — <code>index</code> , <code>data_0</code> ~ <code>data_3</code> ,	<code>ccl_chromium_reader.ccl_chromium_reader</code> (<code>guess_cache_class</code> → <code>ChromiumBlockFileCache</code>) + <code>zstd</code> / <code>brotli</code> / <code>gzip</code> 해제
Local Storage	<code>...\Claude\Local Storage\leveldb\</code>	ccl_chromium_reader
Session Storage	<code>...\Claude\Session Storage\</code>	ccl_chromium_reader
쿠키(앱 전용)	<code>...\Claude\Network\Cookies</code> · Trust Tokens	sqlite3 + DPAPI
기기 식별자	<code>...\Claude\ant-did</code> (48B base64)	텍스트 에디터
계정·디바이스 등록부	<code>...\Claude\ant-device-registry.json</code>	텍스트 에디터
앱 설정/토론	<code>...\Claude\config.json</code>	텍스트 에디터
사용량 타임라인	<code>...\Claude\plan-usage-history.json</code>	텍스트 에디터
MCP 설정	<code>...\Claude\claude_desktop_config.json</code>	텍스트 에디터
산출물 폴더	config의 <code>coworkUserFilesPath</code> 값 (기본 <code>%USERPROFILE%\Claude</code>)	탐색기
로그	<code>...\Claude\logs\</code> main.log · claude.ai-web.log · ssh.log · cowork_vm_node.log · unknown-window.log	텍스트 에디터
로컬 에이전트 세션	<code>...\Claude\local-agent-mode-sessions\<accountUuid>\</code>	탐색기

분석 대상	저장 위치	도구
Claude Code 연 동	...\\Claude\\claude-code\ · claude-code-vm\ · vm_bundles\ · git- worktrees.json	탐색기
기타	...\\Claude\ 내 fcache · sentry\ · window-state.json · Preferences · Local State · DIPS(-wal 3MB) · InterestGroups · SharedStorage · Shared Dictionary · blob_storage\	혼합

```

레코드 총 750건 (live=707, 삭제·tombstone=43) 대화 UUID 단위로 키가 분리됨
키 형식: store:chat-draft:<conversation-uuid>

— 마지막 11개 스냅샷 (약 2초 간격 자동 저장) —
06:55:14 len=41 ...된거고 확실한 거야? ♪ 내가 테스트 해봐야 할 건 뭐고, 더 필요
06:55:16 len=48 ... 거야? ♪ 내가 테스트 해봐야 할 건 뭐고, 더 필요한 건 없는지
06:55:18 len=57 ... 테스트 해봐야 할 건 뭐고, 더 필요한 건 없는지, 그리고 너가
06:55:19 len=58 ...테스트 해봐야 할 건 뭐고, 더 필요한 건 없는지, 그리고 너가
06:55:21 len=60 ...트 해봐야 할 건 뭐고, 더 필요한 건 없는지, 그리고 너가 서준데
06:55:22 len=58 ...테스트 해봐야 할 건 뭐고, 더 필요한 건 없는지, 그리고 너가
06:55:24 len=58 ...테스트 해봐야 할 건 뭐고, 더 필요한 건 없는지, 그리고 너가 써
06:55:26 len=64 ...야 할 건 뭐고, 더 필요한 건 없는지, 그리고 너가 써준것 중 올
06:55:28 len=70 ...뭐고, 더 필요한 건 없는지, 그리고 너가 써준것 중 오류나 오탐은
06:55:30 len=77 ...요한 건 없는지, 그리고 너가 써준것 중 오류나 오탐은 없는지 다시
06:55:32 len=86 ... 그리고 너가 써준것 중 오류나 오탐은 없는지 다시 한 번 더 검즈

```

[1] 앱 IndexedDB store:chat-draft 로 타이핑 과정 재구성
len 값이 41→86 으로 증가하며 문장이 자라나는 것이 보인다.

```

캐시 키 : claude.ai/api/organizations/0eb65233-000c-48b3-9559-f619cf486961/chat_conversation
압축 : zstd 307,417 bytes → 복원 1,360,202 bytes
대화 : uuid=88d8bdc8-f420-4152-9b9c-44f57bab920e
      name='LLM 대화 분석 디지털포렌식 프로젝트 설계'
      model=claude-opus-4-8 created=2026-08-01T16:16:55.477017Z 메시지 115건

— 복원된 메시지 (앞 6건) —
[human ] 2026-08-01T16:16:57 디지털포렌식 프로젝트 할건데 LLM 대화내용 추출 및 분석하는
[assistant] 2026-08-01T16:18:27 디지털 포렌식 프로젝트 설계, 정리해서 파일로 만들어 드릴게
[human ] 2026-08-01T16:20:56 디지털포렌식 프로젝트 할건데 LLM 대화내용 추출 및 분석하는
[assistant] 2026-08-01T16:21:14
[human ] 2026-08-01T16:22:05 디지털포렌식 프로젝트 할건데 LLM 대화내용 추출 및 분석하는

```

[2] 앱 HTTP Cache 에서 대화 전문 복원
zstd 307,417B → 1,360,202B. 대화 제목·모델·생성시각과 함께 human/assistant 메시지 115건이 발신자·시각·본문 단위로 복원
된다.

```

[ant-did] 48 bytes, base64 1줄
raw : N2Y2MGJ1MzI0Mjc4Yi00NzFkLWE0ZWYyZkFhRmRhYjNlMmM5
decode : 7f60bb32-271b-471d-a4ea-c6d1dab3f2c7 <- 기기 고유 UUID

[ant-device-registry.json] 계정별 디바이스 공개키 등록부 (PDF 미수록)
account ce466787-2348-443c-8791-4781066b53e4 -> pk1:a5ea3ad4eb377100134a32333dd98c...
account 95eb3021-873c-495d-999b-21f405a34d2d -> pk1:a9e22af9b438eec4777f24aa9a8ea4...

[HKCU\...\AppModel\SystemAppData\Claude_pzs8xrnjxfjhc\HAM\AU\Claude\W1\LU]
MSIX 앱은 UserAssist 대신 HAM/AUI 에 실행 흔적이 남는다 (PDF 미수록)
PCT = 134303740530235832 => 2026-08-05 12:27:33
PTT = 134294673459306952 => 2026-07-26 00:35:45
(대조) UserAssist 전체 항목 수 = 0 -> 본 시스템에서 UserAssist 는 무효

[plan-usage-history.json] 사용량 샘플 1406건 - 계정(org)별 사용 타임라인 (PDF 미수록)
[ 0] 2026-07-19 10:04:53 org=21ea1d47-6bb8-4c1c-b0ed-4ab21aee18c fh=26 sd=30
[ 1] 2026-07-19 10:09:53 org=21ea1d47-6bb8-4c1c-b0ed-4ab21aee18c fh=27 sd=30
[1404] 2026-08-11 16:36:55 org=0eb65233-000c-48b3-9559-f619cf486961 fh=6 sd=41
[1405] 2026-08-11 16:41:55 org=0eb65233-000c-48b3-9559-f619cf486961 fh=6 sd=41

[logs\main.log] 앱 기동 이벤트 = 실행 타임라인
2026-05-11 14:56:11 [info] Starting app {
  appVersion: '1.6608.2',
  isPackaged: true,
  ... 총 44,571 줄, 마지막 기록 8/11/2026 4:44:29 PM

```

[3] 앱 고유 식별자 · MSIX 실행 흔적 · 사용량 타임라인
ant-did 의 base64 디코드, 계정별 디바이스 등록부, HAM/AUI 의 PCT-PTT, 그리고 1,406건짜리 사용량 타임라인.

▼ 애플리케이션 detail - 1 설치 형태, 2 데이터 루트, 3 IndexedDB

```
PS C:\Users\sol> Get-AppxPackage *Claude* | Select-Object Name, Version, Publisher, PackageFullName, SignatureKind, InstallDate

Name       : Claude
Version    : 1.38996.1.0
Publisher  : CN=Anthropic, PBC, O=Anthropic, PBC, L=San Francisco, S=California, C=US,
            CSDNALNUM83=8666c21, OID.2.5.4.15=Private Organization,
            OID.1.3.6.1.4.1.311.60.2.1.2=Delaware, OID.1.3.6.1.4.1.311.60.2.1.3=US
PackageFullName : Claude_pzs8srxjxfjjc
SignatureKind : Developer
InstallDate :

PS C:\Users\sol> Get-Childitem (Get-AppxPackage *Claude*).InstallLocation | Select Name, Length

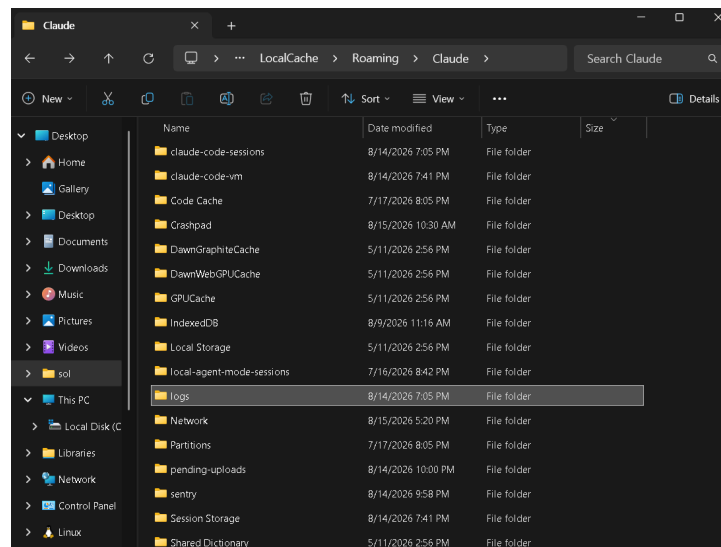
Name      Length
-----
App
Assets
AppxBlockMap.xml 1008759
AppxManifest.xml 7914
AppxSignature.p7x 10547
resconfig.xml    1408
resources.pri    266384
resources.scale-200.pri 87936

PS C:\Users\sol> Get-AppxPackage | Where-Object { $_.Name -like "*Claude*" -or $_.Publisher -like "*Anthropic*" } | Select Name, PackageFullName

Name      PackageFullName
-----
Claude Claude_1.38996.1.0_x64__pzs8srxjxfjjc
```

[그림 1] 설치 형태

Claude/Anthropic 관련 Appx 전체 조회 결과 1건만 검출, 2차 병존 패키지 없음(단일 MSIX).



[그림 2] 데이터 루트

C:\Users\sol\AppData\Local\Packages\Claude_pzs8srxjxfjjc\LocalCache\Roaming\Claude

```
PS C:\Users\sol\Desktop> python -lib.py ".\claude_idb_copy\https_claude.ai_0.indexeddb.leveldb"
TARGET: claud_idb_copy\https_claude.ai_0.indexeddb.leveldb
BLOB : claud_idb_copy\https_claude.ai_0.indexeddb.blob

[A] LevelDB raw keys: 23777 (Live 15775 / deleted 8002)

[B] databases: 4

DB #4 claud-notifications (stores: 1)
- seenMessages total 29 (Live 29 / deleted 0)

DB #3 claud-device-binding (stores: 1)
- device_binding_keys total 0 (Live 0 / deleted 0)

DB #2 frame-mcp-cache (stores: 1)
- cache total 1 (Live 1 / deleted 0)

DB #1 keyval-store (stores: 1)
- keyval total 1393 (Live 1333 / deleted 60)
```

[그림 3] IndexedDB

앱 로컬 저장소 4개 DB에서 사용자 활동 흔적이 확보됨 — keyval-store(1393레코드)는 대화·세션 상태 캐시의 본체, claud-notifications의 seenMessages 29건은 사용자가 실제로 확인한 알림 이력으로 앱 사용 시점 추정에 활용, frame-mcp-cache의 wipedAt 마커는 캐시 초기화 시각을, claud-device-binding 0건은 기기 바인딩 미설정을 각각 증거한다. 특히 LevelDB raw keys 23777건 중 삭제 8002건이 톰스톤으로 잔존해, 논리적으로 삭제된 데이터가 컴팩션 전까지 디스크에 남아 복원 가능성을 실증한다.

▼ 애플리케이션 detail - 4 입력 초안

```
>>> d = [(k, l) for k, l in keys if isinstance(k, str) and k.startswith("store:chat-draft:")]
>>> len(d), sum(1 for _, l in d if l), sum(1 for _, l in d if not l)
(896, 840, 56)
>>> Counter(k.replace("store:chat-draft:", "") for k, _ in d)
Counter({'18d8bdc-b-f420-4152-9b9c-44f57bab920e': 682, 'cse_018NCoc8NMtCL5apFu5Yc68a': 82, 'chorus-unified-composer': 78, 'cse_013K3hrgQW0dubv3j6p7xg3': 49, 'cse_01PtgbJamm5a3D3untLBAZZ': 5})
>>>
```

[그림 4-1] 초안 레코드 총계와 키 분포

store:chat-draft: 접두사를 가진 레코드는 896건(Put 840 / Delete 56)으로, **keyval-store** 전체 1393건의 64%를 차지한다. 키는 5종으로 분리되며 대화 UUID 1종이 682건(76%)을 점유한다. 나머지는 세션 ID **cse_*** 3종과 고정 키 **chorus-unified-composer** 1종으로, 후자는 대화가 생성되기 이전 단계의 입력을 담는다.

```
>>> rec = next(r for r in store.iterate_records(live_only=True) if isinstance(r.key.value, str) and r.key.value.startswith("store:chat-draft:"))
>>> rec.key.value
'store:chat-draft:8d8bdc-b-f420-4152-9b9c-44f57bab920e'
>>> list(rec.value)
['state', 'version', 'updatedAt']
>>> list(rec.value["state"])
['tiptapEditorState', 'attachments', 'files', 'syncSourceUids', 'queuedMessages']
```

[그림 4-2] 값 구조

초안 레코드의 최상위 필드는 **state** / **version** / **updatedAt**이며, 에디터 상태는 **state.tiptapEditorState** 경로에 저장된다. **updatedAt**은 밀리초 단위 epoch로, 자동저장 주기 산출의 근거가 된다.

```
>>> import re
>>> re.findall(b'[-\w]+', list(d.values()))
['state', 'b'tiptapEditorState', 'b'type', 'b'doc', 'b'contentA', 'b'type', 'b'paragraph', 'b'contentA', 'b'type', 'b'text', 'b'text', 'b'attachmentsA', 'b'filesA', 'b'syncSourceUidsA', 'b'queuedMessagesA', 'b'version', 'b'updatedAt']
```

[그림 4-3] 직렬화된 원본 필드명

LevelDB에 저장된 값의 바이트열에서 ASCII 구간만 추출한 결과. **state** → **tiptapEditorState** → **type/doc** → **content** → **paragraph** → **text**로 이어지는 문서 노드 트리와 함께 **attachments**, **files**, **syncSourceUids**, **queuedMessages**, **version**, **updatedAt** 필드가 확인된다. 필드명은 1바이트로 저장되어 ASCII 검색에 그대로 노출되는 반면, 본문 텍스트는 UTF-16LE로 저장된다(그림 4-9 참조).

```
>>> import json, re
>>> txt = lambda v: "".join(re.findall(r"text": "\x" + ((v["text"] if v["text"] else "")) if v["text"] else ""))
>>> = [print(f'{r.key.value["state"]}: {r.value["state"]}') for r in sorted(drafts, key=lambda r: r.key.value["state"])]
271 live=True 8d8bdc-b-f420 'state'
283 live=True 8d8bdc-b-f420 'state'
295 live=True 8d8bdc-b-f420 'state'
307 live=True 8d8bdc-b-f420 'state'
319 live=True 8d8bdc-b-f420 'state'
331 live=True 8d8bdc-b-f420 'state'
343 live=True 8d8bdc-b-f420 'state'
355 live=True 8d8bdc-b-f420 'state'
367 live=True 8d8bdc-b-f420 'state'
379 live=True 8d8bdc-b-f420 'state'
391 live=True 8d8bdc-b-f420 'state'
403 live=True 8d8bdc-b-f420 'state'
415 live=True 8d8bdc-b-f420 'state'
427 live=True 8d8bdc-b-f420 'state'
439 live=True 8d8bdc-b-f420 'state'
451 live=True 8d8bdc-b-f420 'state'
463 live=True 8d8bdc-b-f420 'state'
475 live=False 8d8bdc-b-f420 'state'
477 live=True 8d8bdc-b-f420 'state'
541 live=True 8d8bdc-b-f420 'state'
```

[그림 4-4] 타이핑 과정 복원

동일 키에 대한 저장 버전을 LevelDB 시퀀스 번호 순으로 나열한 결과. 텍스트가 한 글자씩 누적되는 과정이 그대로 복원된다.

tmso (한글 입력 상태에서 영문 자판으로 오입력), **임미징** (한글 IME 조합 중간 상태), **스냅사시** → **스냅삿이** (오타 및 수정)가 모두 개별 레코드로 남아 있어, 최종 전송문뿐 아니라 작성자의 입력 행위 자체가 재구성 가능함을 보여준다.

```
>>> = [print(f'{r.key.value["state"]}: {r.value["state"]}') for r in sorted(drafts, key=lambda r: r.key.value["state"])]
451 live=True 'state'
463 live=True 'state'
475 live=False 'state'
477 live=True 'state'
541 live=True 'state'
553 live=True 'state'
565 live=True 'state'
577 live=True 'state'
589 live=True 'state'
601 live=True 'state'
```

[그림 4-5] 전송 시 tombstone

seq 474에서 값이 빈 Delete 레코드(tombstone)가 기록되고, 직후 seq 477부터 새로운 초안이 시작된다. 이는 메시지 전송 시 초안이 비워지는 동작에 해당한다. 그러나 전송 직전까지의 모든 중간 버전(seq 271~463)은 삭제되지 않고 디스크에 그대로 남아 있어, 사용자 화면에서 사라진 입력이 저장소에서는 유지됨을 보여준다.

```
>>> ts = sorted(r.value["updatedAt"] for r in drafts if isinstance(r.value, dict) and r.value.get("updatedAt"))
>>> diffs = [round((b - a) / 1000, 1) for a, b in zip(ts, ts[1:])]
>>> Counter(diffs).most_common(10)
[(2.0, 145), (2.1, 47), (1.0, 44), (0.9, 43), (0.7, 41), (1.5, 32), (2.2, 32), (0.6, 28), (1.4, 27), (0.8, 26)]
```

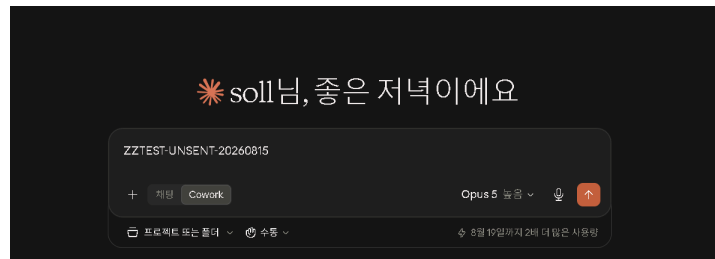
[그림 4-6] 자동저장 주기

`updatedAt` 값의 인접 차분 분포. 2.0초가 145회로 단독 최빈값이며, 이는 약 2초 주기의 자동저장을 뒷받침한다. 2초 미만 구간은 서로 다른 초안 키의 저장 시각이 교차한 결과와 포커스 이동·붙여넣기 등 이벤트성 저장이 혼재한 것으로 해석된다.

```
>>> from datetime import datetime
>>> datetime.fromtimestamp(ts[0]/1000), datetime.fromtimestamp(ts[-1]/1000)
(datetime.datetime(2026, 8, 5, 12, 44, 19, 290000), datetime.datetime(2026, 8, 15, 17, 22, 58, 644000))
>>> import os, datetime as dt
>>> base = os.environ["LOCALAPPDATA"] + r"\Packages\Claude_pzs8srxjxfjje"
>>> dt.datetime.fromtimestamp(os.path.getctime(base))
datetime.datetime(2026, 5, 11, 14, 56, 9, 493420)
```

[그림 4-7] 잔존 구간과 컨테이너 생성일

초안 기록의 잔존 구간은 2026-08-05 12:44:19 ~ 2026-08-15 17:22:58이다. 패키지 컨테이너 생성일은 2026-05-11 14:56이며, `logs\main.log` 는 2026-05-11 14:56:11 최초 실행부터 연속 기록되어 있어 이 설치본이 그 이전부터 사용되어 왔음을 보여준다. 초기 검토에서 2026-07-26 18:40의 IndexedDB 재생성을 하한의 원인으로 추정하였으나, 로그 확인 결과 해당 시각의 이벤트는 약 17시간 58분의 절전 상태에서 복귀한 것(`[event-loop-stall] ... likely sleep: power_event, system woke`)이며 데이터 초기화가 아니었다. 또한 로그에는 2026-07-16 20:42:36에 계정 식별자가 전환된 기록이 있다. 현 시점에서 초안 기록이 2026-08-05부터 시작하는 원인은 **확정되지 않았으며**, LevelDB 컴팩션에 의한 정리, 계정 전환에 따른 저장소 분리, 해당 기간 데스크톱 앱에서의 작성 부재 등이 후보로 남아 있다.



[그림 4-8] 미전송 마커 입력 상태

대조 실험 1단계. 고유 마커 문자열을 입력창에 입력한 뒤 전송하지 않은 상태. 이후 앱을 종료하고 데이터 사본을 생성하여 해당 문자열의 디스크 잔존 여부를 확인한다.

```
>>> len([r for r in ldb.RawLevelDb(P2).iterate_records_raw() if b"ZZTEST-UNSENT-20260815" in r.value])
0
>>> pats = [b"ZZTEST", "ZZTEST".encode("utf-16-le")]
>>> [(os.path.basename(d), len([r for r in ldb.RawLevelDb(d).iterate_records_raw() if any(x in r.value for x in pats)])) for d in glob.glob(r"C:\Users\sol\Desktop\ldb2\*.leveldb")]
[('https_claude.ai_0_indexeddb.leveldb', 3)]
```

[그림 4-9] ASCII 검색

동일 데이터에 대한 두 검색의 결과 차이. ASCII 바이트열로 검색하면 0건이나, UTF-16LE 패턴을 포함하면 3건이 검출된다. 초안 본문이 UTF-16LE로 직렬화되기 때문이며, ASCII 그래프만 수행할 경우 "데이터 없음"이라는 잘못된 결론에 도달할 수 있다.

```
>>> hits = [r for r in ldb.RawLevelDb(P2).iterate_records_raw() if any(x in r.value for x in pats)]
>>> [(r.state, len(r.value), os.path.basename(str(r.origin_file))) for r in hits]
[(<KeyState.Live: 1>, 284, '000004.log'), (<KeyState.Live: 1>, 284, '000004.log'), (<KeyState.Live: 1>, 378, '000004.log')]
>>> [ldb.IdbKey(r.key[len(pre2):]).value for r in hits if r.key.startswith(pre2)]
['store:chat-draft:chorus-unified-composer', 'store:chat-draft:chorus-unified-composer']
>>> [r.key.startswith(pre2) for r in hits]
[True, True, False]
>>> i16 = "ZZTEST".encode("utf-16-le")
>>> [r.value[max(0, r.value.find(i16)-60):r.value.find(i16)+160].decode("utf-16-le", "ignore") for r in hits if i16 in r.value]
['指湯整瑛\&000004.log', '指湯整瑛\&000004.log', '指湯整瑛\&000004.log']
```

[그림 4-10] 미전송 입력 검출

전송하지 않은 입력이 디스크에 기록되었음을 보여주는 검출 결과. 마커 `ZZTEST-UNSENT-20260815` 을 입력창에 입력만 하고 전송하지 않은 상태에서 앱을 종료한 뒤, 생성한 사본을 검색한 결과 3건이 검출되었다. 3건 모두 `KeyState.Live` (Put 레코드)이며 현재 쓰기 로그인 `000004.log` 에 위치한다. 그중 2건(각 284바이트)은 오브젝트 스토어 키 `store:chat-draft:chorus-unified-composer` 에 속한다 — 대화 화가 아직 생성되지 않은 상태의 입력이므로 대화 UUID가 아닌 고정 키에 저장된 것이다. 나머지 1건(378바이트)은 해당 키 접두사에 속하지 않으며, IndexedDB 내부 `scopes/undo` 로그에 남은 사본이다(그림 4-11).

UTF-16LE로 디코딩하면 마커 문자열이 잘리지 않고 온전히 보존되어 있음이 확인된다. 주변의 판독 불가 문자(`指湯整瑛`, `潤瑛瑛瑛` 등)는 1바이트로 저장된 필드명을 UTF-16으로 잘못 읽은 결과이며, 같은 값에서 ASCII 구간만 추출하면 `state` → `tipTapEditorState` → `type/doc` → `content` → `paragraph` → `text` 로 이어지는 문서 노드 트리와 `attachments`, `files`, `syncSourceUuid`, `queuedMessages`,

`version`, `updatedAt` 필드가 드러난다. 즉 본문 텍스트는 UTF-16LE, 필드명은 1바이트로 혼재 저장되며, 이 때문에 ASCII 단독 검색으로는 본문이 검출되지 않는다.

```
>>> bad = [r for r in hits if not r.key.startswith(pre2)][0]
>>> bad_key[10], bad_state, len(bad.value), os.path.basename(str(bad.origin_file))
(b'\x00\x00\x00\x00\x02\x02\x02\x02\x04\x00\x7f', '<KeyState.Live: 1>', 378, '000004.log')
>>> scopes = [r for r in ldb.RawLevelDb(P2).iterate_records_raw() if r.key.startswith(b"\x00\x00\x00\x02")]
>>> len(scopes)
21802
>>> len([r for r in scopes if b"tipTapEditorState" in r.value])
883
```

[그림 4-11] undo 로그 중복 기록

오브젝트 스토어에 속하지 않은 나머지 1건의 키는 `00 00 00 00 32 02 <scope varint> 00` 으로, IndexedDB 내부 scopes 영역의 undo 태스크 로그에 해당한다(트랜잭션 롤백용 이전 이미지). 값 크기가 오브젝트 스토어 레코드(284바이트)보다 큰 378바이트인 것도 undo 레코드가 원본을 감싸기 때문이다.

이 사본은 IndexedDB API로 접근·삭제할 수 없으며, LevelDB 컴팩션 시점까지 잔존한다. 그 결과 두 가지 함의가 발생한다. 첫째, 앱이 초안을 삭제하더라도 undo 로그의 사본은 남으므로 **삭제 동작이 데이터의 실제 소거를 보장하지 않는다**. 둘째, 조사 시 오브젝트 스토어를 정상 경로로 열거하는 방식(`iterate_records`)만 사용하면 이 사본은 결과에 포함되지 않는다 — 본 실험에서도 검출된 3건 중 1건이 이에 해당했다. 따라서 **RawLevelDb 수준의 전수 스캔과 scopes 접두사 (`00 00 00 00 32`) 별도 확인이 병행되어야 누락이 없다**.

```
>>> w3 = ldb.WrappedIndexDB(P)
>>> s3 = w3["keyval-store"]["keyval"]
>>> next(r for r in s3.iterate_records(Live_only=True) if isinstance(r.key.value, str) and r.key.value.startswith("store:chat-draft:"))
IndexedDbRecord(owner=<ccl_chromium_reader.ccl_chromium_indexeddb.IndexedDb object at 0x000002BE9B864490>, db_id=1, obj_store_id=1, key=<ldbkey store:chat-draft:88d8bdcf-f420-4152-9b9c-44f57bab920e>, value={
  'state': {'tipTapEditorState': {'type': 'doc', 'content': [{'type': 'paragraph', 'content': [{'type': 'text', 'text': 'msc'}]}]}], 'attachments': [], 'files': [], 'syncSourceUids': [], 'queuedMessages': []}, 'version': 2, 'updatedAt': 1785901459290.0}, is_live=True, ldb_seq_no=271, origin_file=WindowsPath('C:/Users/sol/Desktop/claude_idb_copy/https_claude.ai_0_indexeddb.leveldb/000003.log'), external_value_path=None)
```

[그림 4-12] 대용량 초안의 외부 저장

일정 크기를 넘는 초안 값은 LevelDB 내부가 아니라 `https_claude.ai_0.indexeddb.blob` 디렉터리에 개별 파일로 분리 저장된다. blob 경로를 지정하지 않고 읽으면 해당 레코드에서 오류가 발생하며, 이는 IndexedDB 디렉터리만 수집할 경우 초안 일부가 누락될 수 있음을 의미한다.

▼ 애플리케이션 detail - 5 첨부 blob

```
[r.key.value[22], "Put" if r.is_live else "Delete", r.ldb_seq_no, type(r.value).__name__, r.value) for r in ab]
[('4c8955ed-270a-4eb7-9aee-e3e2f360d200', 'Put', 5341, 'BlobIndex', 'BlobIndex(index_type=<BlobIndexType.File: 1>, index_id=0)', ('4c8955ed-270a-4eb7-9aee-e3e2f360d200', 'Delete', 5879, 'NoneType', None), ('07a06376-3a5f-44f4-8d08-2d207b90f9f', 'Put', 12242, 'BlobIndex', 'BlobIndex(index_type=<BlobIndexType.File: 1>, index_id=0)', ('07a06376-3a5f-44f4-8d08-2d207b90f9f', 'Delete', 12930, 'NoneType', None))
[(m.blob_number, m.mime_type, m.size, m.file_name, m.last_modified) for r in ab if hasattr(r.value, "index_id") for m in [r.resolve_blob_index(r.value)]]
[('image/png', 129268, '1786241762071_image.png', datetime.datetime(2026, 8, 9, 2, 16, 2, 68000)), (101, 'image/png', 32207, '1786712338700_image.png', datetime.datetime(2026, 8, 14, 12, 58, 58, 699000))]
[(f'{r.db_id:x}\\{m.blob_number>8:02x}\\{m.blob_number:x}', os.path.exists(os.path.join(B, f'{r.db_id:x}', f'{m.blob_number>8:02x}', f'{m.blob_number:x}')) for r in ab if hasattr(r.value, "index_id") for m in [r.resolve_blob_index(r.value)]]
[(f'\\{00\\2', False), (f'\\{00\\65', False)]
>>> [p.replace(B, ""), os.path.getsize(p)] for p in glob.glob(B + r"\\*.*", recursive=True) if os.path.isfile(p)]
[(f'\\{01\\ldb', 591133)]
[(os.path.getsize(p), DT.fromtimestamp(int(re.search(r"-(\\d{13})", p).group(1))/1000).strftime("%m-%d %H:%M:%S"), os.path.basename(p)[-26:]) for p in sorted(glob.glob(root + r"\\pending-uploads\\*"))]
[(64089, '08-15 17:44:27', '25-1786783467528_image.png'), (32207, '08-14 21:58:58', '08-1786712338700_image.png'), (320218, '08-15 17:35:37', '40-1786782997072_image.png'), (60404, '08-15 17:41:54', '38-1786783314764_image.png'), (131773, '08-15 21:13:19', '5e-1786795999234_image.png')]
>>> import shutil; shutil.copy([p for p in glob.glob(root + r"\\pending-uploads\\*") if "1786712338700" in p][0], r"C:/Users/sol/Desktop/recovered.png")
>>> os.path.isdir(root + r"\\File System"), os.path.isdir(root + r"\\WebStorage")
(False, True)
```

[그림 5] - 대화 첨부 파일의 저장·삭제·잔존을 다섯 단계로 확인한 결과.

① **키 구성** — 첨부는 `store:attachment-blob:<uuid>` 키로 저장된다. 레코드는 총 4건, UUID는 2종(`4c8955ed...d200`, `07a06376...0f7f`)이며 각 UUID마다 Put 1건과 Delete 1건이 시퀀스 순서대로 존재한다(5341→5879, 12242→12930). 두 첨부 모두 등록 후 삭제되었으므로 현재 유효한 첨부는 없다. Put 레코드의 값은 `BlobIndex(index_type=File, index_id=0)` 으로 파일 데이터가 아닌 **외부 blob 포인터**이고, Delete 레코드의 값은 `None` 이다.

② **메타데이터** — `resolve_blob_index()` 로 포인터를 해석하면 blob 번호·MIME·바이트 수·원본 파일명·수정시각이 복원된다. 첫째는 blob #2, `image/png`, 129,268바이트, `1786241762071_image.png`, 2026-08-09 11:16:02 KST. 둘째는 blob #101, `image/png`, 32,207바이트, `1786712338700_image.png`, 2026-08-14 21:58:58 KST. 파일명에 밀리초 epoch가 포함되어 업로드 시각을 직접 얻을 수 있으며, 라이브러리가 출력하는 시각은 UTC 기준이다.

③ **실물 부재** — 경로 규칙 `{db}\{번호>8:02x}\{번호:x}` 가 가리키는 `\\{00\\2` 와 `\\{00\\65` 는 모두 `False`, 즉 파일이 존재하지 않는다. 첨부 삭제 시 blob 실물은 디스크에서 실제로 제거된다. 초안 텍스트가 삭제 후에도 이전 버전 전체가 잔존하는 것과 대조되는 지점이다. blob 디렉터리에 남은 유일한 파일 `\\{01\\ldb` (591,133바이트)는 두 첨부 어느 쪽도 아니다.

④ **스테이징 원본과 복원** — 업로드 스테이징 폴더 `pending-uploads` 에는 2026-08-14 ~ 08-15 이미지 5건이 정리 없이 누적되어 있다. 이 중 32,207바이트 파일은 ②에서 확인한 둘째 첨부 크기·파일명과 정확히 일치한다. 해당 파일을 복사하여 `recovered.png` 로 복원하는 데 성공했다. IndexedDB에서 삭제된 첨부 원본이 별도 경로에 열람 가능한 상태로 남아 있는 것이다. 반면 첫째 첨부(2026-08-09)는 이 폴더에 없어 복원되지 않았다.

⑤ **OPFS 미사용** — 프로필 하위에 **File System** 디렉터리가 존재하지 않아(**False**) OPFS는 사용되지 않으며, 함께 확인된 **WebStorage (True)**는 저장소 쿼터 관리용으로 OPFS와 무관하다. 따라서 첨부 저장 경로는 IndexedDB Blob 방식임이 확인된다.

종합하면, 첨부 삭제는 blob 실패까지 제거하지만 IndexedDB에 메타데이터가 남아 **무엇이 언제 첨부되었는지 특정할 수 있고**, 스테이징 폴더에 원본이 남는 경우 **내용까지 복원할 수 있다**.

▼ 애플리케이션 detail - 6 HTTP Cache

저장소	남는 것	규모
IndexedDB store:chat-draft:88d8bdc...	전송 전 타이핑 중간 버전	682 레코드
HTTP 캐시 — 상세 응답	대화 전문	117 메시지 / 82,542자 / 1.36MB
HTTP 캐시 — 목록 응답	제목 + 요약	4,156자

- 캐시는 초안보다 **오래된 이력**을 보유한다. 입력 초안은 2026-08-05 이후만 남지만, 캐시 목록에는 2026-05-29-06-21 생성 대화의 제목·요약·모델이 있다.
- 대화 목록 응답(`chat_conversations_v2`, zstd 10,760B → 32,835B)에는 전문이 없는 대신 대화 11건의 **제목(6~30자)**과 **요약문(0~4,156자, 평균 약 1,720자)**, 모델, 생성 시각, `session_id`, `user_uuid`가 담긴다. 전문이 추출된 뒤에도 이 요약은 남는다.

```
>>> len(keys), sum(1 for k in keys if "claude" in k), sum(1 for k in keys if "/api/" in k)
(1181, 1158, 180)
>>> orgs = sorted([m.group(1) for k in keys for m in [re.search(r"/organizations/([0-9a-f-]{36})", k) if m]])
>>> len(orgs), [(o[:8], sum(1 for k in keys if o in k)) for o in orgs]
(1, [(0eb65233, 172)])
>>> collections.Counter(re.sub(r"[0-9a-f-]{36}", "<uuid>", k.split("?")[0].split("/api/")[1]) for k in keys if
f"/api/" in k).most_common(15)
[('organizations/<uuid>/files/<uuid>/contents', 5), ('organizations/<uuid>/preview', 5), ('organizations/<uuid>/
chat_conversations_v2', 4), ('directory/servers', 4), ('organizations/<uuid>/plugins/list-plugins', 3), ('org
anizations/<uuid>/projects', 3), ('organizations/<uuid>/usage', 2), ('organizations/<uuid>/skills/download-dot
-skill-file', 2), ('organizations/<uuid>/experiences/claude_web', 2), ('organizations/<uuid>/skills/list-skill
s', 2), ('organizations/<uuid>/cowork/messages/msg_011Ce4kdKvDmUxVrst7ytR/safety_flags', 1), ('organizations
/<uuid>/cowork/messages/msg_011Ce4hJ82xorm3xjDUBTh4G/safety_flags', 1), ('organizations/<uuid>/cowork/messages
/msg_011Ce4DfvzJ9joeAHxv3VFnD/safety_flags', 1), ('organizations/<uuid>/cowork_settings', 1), ('organizations/
<uuid>/projects/count', 1)]
```

[그림 6-1] HTTP 캐시의 규모와 캐시되는 API 응답

Chromium blockfile 캐시를 `ccl_chromium_reader`로 판독한 결과, 엔트리는 총 1,181건이며 그중 1,158건이 `claude` 도메인, 180건이 `/api/` 응답이다. 조직 UUID는 `0eb65233...` 1개(172건). 캐시되는 응답에는 대화 목록·상세뿐 아니라 파일 내용과 미리보기, 아티팩트 버전, 플러그인·스킬 목록, 사용량, 결제수단, Cowork 설정 및 메시지별 안전성 판정 결과가 포함된다.

```
>>> [k.split("/api/")[1][:60] for k in keys if "chat_conversations" in k]
['organizations/0eb65233-000c-48b3-9559-f619cf486961/chat_conv', 'organizations/0eb65233-000c-48b3-9559-f619cf
486961/chat_conv', 'organizations/0eb65233-000c-48b3-9559-f619cf486961/chat_conv', 'organizations/0eb65233-000
c-48b3-9559-f619cf486961/chat_conv']
>>> [k.split("/api/")[1][:60] for k in keys2 if "chat_conversations" in k]
['organizations/0eb65233-000c-48b3-9559-f619cf486961/chat_conv', 'organizations/0eb65233-000c-48b3-9559-f619cf
486961/chat_conv', 'organizations/0eb65233-000c-48b3-9559-f619cf486961/chat_conv', 'organizations/0eb65233-000
c-48b3-9559-f619cf486961/chat_conv', 'organizations/0eb65233-000c-48b3-9559-f619cf486961/chat_conv']
>>> [k.split("/api/")[1][:95] for k in sorted(set(keys2) - set(keys)) if "chat_conversations/" in k]
['organizations/0eb65233-000c-48b3-9559-f619cf486961/chat_conversations/88d8bdc-b-f420-4152-9b9c-4']
```

[그림 6-2] 대화 열람에 따른 상세 응답 캐시 생성

대화 열람 전 스냅샷에는 목록 엔드포인트(`chat_conversations_v2`)만 존재했으나, 앱에서 해당 대화를 열람한 직후 재수집한 사본에는 `chat_conversations/88d8bdc...?tree=True&rendering_mode=messages&render_all_tools=true&consistency=strong` 키가 새로 나타난다. 두 사본의 키 집합 차분으로 확인된다. 이전 스냅샷에서 상세 응답이 보이지 않았던 것은 기능 변경이 아니라 캐시 추출의 결과이며, 사용자가 대화를 다시 열면 전문이 재차 디스크에 기록된다.

```
>>> k2 = [k for k in keys2 if "chat_conversations/" in k][0]
>>> m2 = cache2.get_metadata(k2)[0]
>>> d2 = cache2.get_cachefile(k2)[0]
>>> [a for a in m2.http_header_attributes if a[0] in ("content-type", "content-encoding", "date")]
[('date', 'Sat, 15 Aug 2026 23:50:46 GMT'), ('content-type', 'application/json'), ('content-encoding', 'zstd')]
>>> raw2 = unz(d2)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'unz' is not defined
>>> len(d2), len(raw2), round(len(raw2)/len(d2), 2)
(308680, 1363887, 4.42)
>>> j2 = json.loads(raw2)
>>> j2["uuid"], j2["model"]
('80d6dcd4-f40d-4152-9a9c-44f57b9b529e', 'claude-opus-4')
>>> j2["created_at"][:19], len(j2["chat_messages"]), list(j2["chat_messages"])[0]
('2026-08-01T16:16:55', 117, {'uid', 'text', 'content', 'sender', 'index', 'created_at', 'updated_at', 'input_mode', 'truncated', 'attachments', 'files', 'sync_sources', 'parent_message_uuid'})
>>> tot = lambda m: sum(len(b.get("text") or "") for b in m.get("content", []))
>>> [m["sender"], m["created_at"][:19], tot(m) for m in j2["chat_messages"][:10]]
('human', '2026-08-01T16:16:57', 215), ('assistant', '2026-08-01T16:18:27', 35), ('human', '2026-08-01T16:20:56', 309), ('assistant', '2026-08-01T16:21:14', 0), ('human', '2026-08-01T16:22:05', 343), ('assistant', '2026-08-01T16:26:21', 802), ('human', '2026-08-01T16:32:41', 120), ('assistant', '2026-08-01T16:37:00', 987), ('human', '2026-08-01T16:43:32', 158), ('assistant', '2026-08-01T16:47:02', 1041)]
>>> len(j2["chat_messages"]), sum(tot(m) for m in j2["chat_messages"])
(117, 82542)
```

[그림 6-3] 대화 전문의 복원

해당 응답은 `content-type: application/json`, `content-encoding: zstd` 로 저장되어 있으며 308,680바이트가 1,363,887바이트로 해제된다(약 4.42배). 파싱 결과 대화 `88d8bdc...` (모델 `claude-opus-4-8`, 생성 2026-08-01T16:16:55)의 **메시지 117건**이 확인되고, 각 메시지는 `uuid`, `sender`, `created_at`, `content`, `parent_message_uuid` 등 13개 필드를 갖는다. 본문은 최상위 `text` 가 아닌 `content` 블록 배열에 담기며, 전체 본문 길이는 **82,542자**다. 발신자 구분과 ISO8601 시각이 함께 남아 대화 전체를 시간 순으로 재구성할 수 있다. 앞부분 human 메시지 3건은 도입부가 동일하고 길이만 215 → 309 → 343자로 증가하여, 동일 질문을 수정·보강해 재전송한 이력까지 드러난다.

* Chromium은 응답을 여러 zstd 프레임으로 나눠 저장하므로 zstd 패키지의 단순 decompress()는 실패한다(length mismatch -> decomp 0 != 131072). zstandard 패키지의 stream_reader(..., read_across_frames=True) 로 프레임을 가로질러 읽어야 한다.

```
>>> [(m['sender'], m['created_at'][:19], tot(m) for m in j2["chat_messages"][:10])
[('human', '2026-08-01T16:16:57', 215), ('assistant', '2026-08-01T16:18:27', 35), ('human', '2026-08-01T16:20:
56', 399), ('assistant', '2026-08-01T16:21:14', 0), ('human', '2026-08-01T16:22:05', 343), ('assistant', '2026
-08-01T16:26:21', 802), ('human', '2026-08-01T16:32:41', 120), ('assistant', '2026-08-01T16:37:00', 987), ('hu
man', '2026-08-01T16:43:32', 158), ('assistant', '2026-08-01T16:47:02', 1041)]
>>> len(j2["chat_messages"]), sum(tot(m) for m in j2["chat_messages"])
(117, 82542)
>>> [(m['sender'], (m.get("content") or [{}])[0].get("text", "")[:60]) for m in j2["chat_messages"][:5]]
[('human', '[REDACTED]'), ('human', '[REDACTED]'), ('assistant', '[REDACTED]'), ('assistant', '[REDACTED]'), ('human', '[REDACTED]')]
[('human', '[REDACTED]', ai'), ('assistant', '[REDACTED]', ai'), ('human', '[REDACTED]', ai'), ('assistant', '[REDACTED]', ai')]
```

[그림 6-4] 복원된 메시지 본문

복원된 **content** 블록에서 추출한 실제 대화 본문. 발신자 구분과 함께 사용자가 입력한 문장이 평문으로 저장되어 있음을 보여준다.